

COMP101

Introduction to Programming

2025-26

Lecture-25

2-d List Create Numbers

2-d list review

List dimension = 4 x 7 (4 rows, 7 columns)
(there are 28 elements)

cols	[0]	[1]	[2]	[3]	[4]	[5]	[6]
rows							
[0]	0,0	0,1	0,2	0,3	0,4	0,5	0,6
[1]	1,0	1,1	1,2	1,3	1,4	1,5	1,6
[2]	2,0	2,1	2,2	2,3	2,4	2,5	2,6
[3]	3,0	3,1	3,2	3,3	3,4	3,5	3,6

‘Dimension’ – the size of the 2-d list

‘Element’ - a cell accessed via two indexes in ‘**row-column order**’

‘Addressing’ - first element is [0,0] row=0, col=0, last element is [3,6]

‘Homogenous’ - elements hold the same data-type

2-d list: Create

Two ways to create a 2d list

needs more technique than for a 1-d list

1. Statically

- values are created by the programmer

2. Automatically

- values are created by the software
 - list comprehension (good for large lists)
 - generator (good for very large lists)
 - dynamically don't use (side effects)

2-d list: Create different types

- null list
 - all elements are zero: Most common initialisation
- unary list
 - all elements are 1
- dense list
 - all or most elements have a value
- sparse list
 - some elements have a value, most elements are zero
- diagonal list
 - elements on the main diagonal have same value

Create statically: 2-d null list from two 1-d lists

#create empty (2x2) for 2 rows, 2 columns

```
null_list_1 = [ [0], [0] ]
```

```
num_list_2 = [ [0], [0] ]
```

```
num_list_both = [num_list_1, num_list_2]
```

```
print (num_list_both)
```

```
[ [0,0], [0,0] ]
```

looks like:

0	0
0	0

Not the best way to do this

Create statically: null – better method

#create empty (2x2) for 2 rows, 2 columns

```
arrayNull = [ [0,0], [0,0] ]
```

```
print (arrayNull)
```

```
[ [0,0], [0,0] ]
```

Looks like:

0	0
0	0

Create statically: null – best method

#null list

```
list_3x4_null = [  
    [0,0,0,0],  
    [0,0,0,0],  
    [0,0,0,0]  
]
```

A 'null list' is one with every element set to zero
This a common initialisation

Coding in this format makes it look more natural

Note: No comma here

```
print (list_3x4_null)
```

```
[ [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0] ]
```

Create statically: populated from 1-d lists

#create two separate lists

```
num_list_1 = [1,2,3]
```

```
num_list_2 = [4,5,6]
```

#assign them both to a new list

```
num_list_both = [num_list_1, num_list_2]
```

```
print (num_list_both)
```

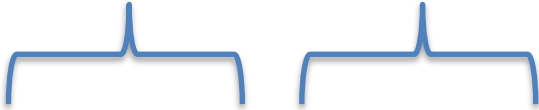
Not the best way to do this

```
[ [1, 2, 3], [4, 5, 6] ]
```


Create statically: populated – better method

Create a 2x3 list (2 rows x 3 columns)

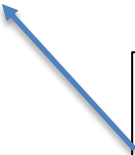
The inner lists are the rows: Two inner lists means two rows



```
list_2x3 = [ [1, 2, 3], [4, 5, 6] ]
```

```
print (list_2x3)
```

```
[ [1, 2, 3], [4, 5, 6] ]
```



Data inside the 'inner lists' are the values of the elements – forming the columns:

Three values in each inner list means three columns

1	2	3
4	5	6

Create statically: populated – better method

```
list_3x3 = [ [1, 2, 3], [4], [5, 6,] ]  
print (list_3x3)
```

column index →

[0] [1] [2] [0] [0] [1]

[[1, 2, 3], [4], [5, 6]]



row index →

[0] [1] [2]

1 2 3

4

5 6

Rows/columns don't have to have the same number of elements in them:

Elements with no value default to null.

This is not a 'sparse list' as the majority of elements are not null

The 'array dimension' here is 3x3 - dimension is determined by the biggest row or column

Create statically: populated – better method

#create populated 2x2

```
List_2x2 = [ [1,2], [3,4] ]
```

```
print (list_2x2)
```

```
[ [1,2], [3,4] ]
```

1	2
3	4

#create populated 3x3

```
List_3x3 = [ [1,2,3], [4,5,6], [7,8,9] ]
```

```
print (list_3x3)
```

```
[ [1,2,3], [4,5,6], [7,8,9] ]
```

1	2	3
4	5	6
7	8	9

Create statically: populated – best method

```
list_3x4 = [
```

```
    [1,2,3,4],
```

```
    [5,6,7,8],
```

```
    [9,10,11,12]
```

```
]
```

```
print (list_3x4)
```

```
[ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12] ]
```

Coding in this format makes it look more natural

Note: No comma here

Create statically: unary

#unary list

```
list_3x4 = [  
    [1,1,1,1],  
    [1,1,1,1],  
    [1,1,1,1]  
]
```

A 'unary list' is one with every element set to 1

```
print (list_3x4)
```

```
[ [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1] ]
```

Create statically: sparse

#sparse list

```
list_3x4 = [  
    [0,0,1,0],  
    [0,2,0,0],  
    [0,0,3,0]  
    ]
```

A 'sparse list' is one with majority of elements set to zero

```
print (list_3x4)
```

```
[ [0, 0, 1, 0], [0, 2, 0, 0], [0, 0, 3, 0] ]
```

Create automatically: list comprehension null list

#static assignment – or can prompt for input

num_of_rows = 4

num_of_cols = 4

#the first 'for' works its way down the rows

#the second 'for' works its way across the rows (the columns)

list_comp = [[0 for row in range (num_of_rows)] for col in range (num_of_cols)]

print(list_comp)

[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]

Good for larger lists

Better method than dynamic (see later)

By-passes the '[0] * col' side-effect

Create automatically: generator null list - best method

```
num_of_rows = 4  
num_of_cols = 4
```

Best method for very large lists

```
list_gen = [ [0] * num_of_cols for i in range(num_of_rows) ]
```

```
print (list_gen)
```

```
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

Create a 4 x 4 null list

In the generator, each element is set to 0 with `[0]*`

Each element is an 'iterator' and created independently from the others.

`[0] * item` is constructed (`num_of_rows`) times – here it is four times

– no reference occurs (no side-effect)

Create Automatically Diagonal population

#create 4x4 array – list comprehension

```
list_diag_4x4 = [[0 for row in range (4)] for col in range (4)]
```

#populate the diagonal with 1's from [0][0] to [3][3]

```
for i in range(4):
```

```
    list_diag_4x4[i][i] = 1
```

#output the matrix

#nested loop - but not nested from above

```
for row in list_diag_4x4:
```

```
    for column in row:
```

#without end=" ", output is one line down the screen

```
    print(column,end=" ")
```

#exit inner loop, o/p shape of list

```
    print()
```

#exit outer loop

```
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
```

Diagonal population has elements all of the same value on the main diagonal;

Used on square matrix:
Can make computation easier and less computationally expensive

Create automatically - dynamically – do not use

```
row = 3
```

```
col = 3
```

```
list_null = [[0] * col] * row
```

```
print (list_null)
```

```
[0, 0, 0], [0, 0, 0], [0, 0, 0]
```

Don't use this method to create a null list

[[0] * col] returns a reference to the list, it doesn't return the list.

Gives a null matrix but if we change [0] [0] =5 we change all of column [0]

```
list_null [0][0] = 5
```

we expect

but actual is

```
5 0 0 0
```

```
5 0 0 0
```

```
0 0 0 0
```

```
5 0 0 0
```

```
0 0 0 0
```

```
5 0 0 0
```